



Numerical Kernels for Dynamic Homotopy Power Flow

My contribution to the team project

SADMAN

Numerical kernels · Section 4.4 · Figure 5 · Richardson control

Based on: Lima-Silva & Freitas, Energies 17(18):4642, 2024

Team: Jonayed · Apurbo · Hasbi · Sizan · Sadman

The power flow problem and why it is hard

Lima-Silva and Freitas, Energies 17(18):4642, 2024

- Power flow solves $g(x) = 0$. The state x holds every bus voltage angle and magnitude.
- Newton-Raphson from a flat start ($V = 1$ pu, angle = 0) diverges on large, ill-conditioned grids.
- The paper reaches the solution in two stages.
- First stage. Build the homotopy $G(x, t) = t g(x) + (1 - t) K (x - x_0)$.
- Integrate the path from $t = 0$ (flat start) to $t = 1$ (the real problem) with FE, RK2 or backward Euler.
- Second stage. Give the endpoint to NR or FDXB for the final iterations.

The paper's headline claim

Backward Euler with only two path points makes NR converge on grids up to 109,272 buses.

What the team reproduced and improved

All nine test systems, every table and figure. Ten changes tested on 34 grids.

What I built

Numerical kernels: linear algebra, step control, tests and tools

1

Core code

LU counter, result types, shared plot style

`linalg.py · results.py · plots.py`

2

Paper results

Section 4.4 pathways and Figure 5 time step

`exp05_sec44_pathways.py · exp08_fig5.py`

3

Improvements

Richardson step control and scaled homotopy

`improvements/path.py · improvements/homotopies.py`

4

From scratch

Gauss elimination and LU with partial pivoting

`improvements/gauss.py`

5

Command line

Run the paper's method with any switch on any grid

`improvements/__main__.py`

6

Report

Shared style plus four figures of the report

`style.py · spy · richardson_path · sec44_grid ·
fig5_tradeoff`

Sparse LU factorization counter

dynhomotopy/linalg.py — the single cost metric of the project

- The paper counts one LU factorization per path point.
- I wrap SciPy's sparse LU (splu) in a small factorize function.
- One LUCounter dataclass records every call.
- Every solver in the project shares the same counter.
- So the cost measure is honest and consistent across NR, FDXB and the homotopy.
- A singular matrix raises SingularMatrixError. This lets adaptive rules retry.

```
@dataclass
class LUCounter:
    factorizations: int = 0

def factorize(a, counter=None):
    if counter is not None:
        counter.factorizations += 1
    try:
        lu = spla.splu(sp.csc_matrix(a))
    except RuntimeError as exc:
        raise SingularMatrixError(str(exc)) from exc
    return lu.solve

def solve(a, b, counter=None):
    return factorize(a, counter)(b)
```

One result type for every solver

dynhomotopy/results.py — SolveResult, Trajectory, HybridResult

SolveResult

- method, converged, iterations
- x, norms (per iteration)
- factorizations, time
- states (optional)

Returned by NR, FDXB and GSH-NR.

Trajectory

- method, K
- times, states, norms
- factorizations, time, failed
- property x_final

Returned by the homotopy path followers.

HybridResult

- trajectory
- refine (SolveResult or None)
- property converged
- property factorizations

Combines a Trajectory with the final refinement.

Shared plot style

experiments/plots.py and report/scripts/style.py

- One place for colours, fonts and figure sizes.
- Every experiment plot and every report figure imports from it.
- So all figures in the report look like one document.
- The report copy adds save helpers for PDF and for generated LaTeX rows.

What the style file gives

Colour cycle	One brand of blues, greens, oranges, reds
Font	Consistent size for axis labels and legends
Figure size	Fixed width for one-column and two-column plots
Grid	Thin grey grid, no top or right spines
Save helper	PDF with tight bounding box
Table helper	Writes LaTeX row files for the report

Larger first step with fewer path points

exp05_sec44_pathways.py — dt0 up to 0.1 with delta held at 0.02

- The paper tries $dt0 = 0.05$ with $K = 1e-3$, then $dt0 = 0.1$ with $K = 2e-3$.
- $\delta = K / dt0$ stays at 0.02 in every run.
- Nine paths with 3, 4 or 5 points. Three integrators: FE, RK2 and BE.
- Every cell reports NR and FDXB iterations, or 'fail'.
- Backward Euler works on almost every path, as the paper reports.
- New: FE and RK2 also work on some 4-point paths.
- New: BE + NR fails on case36964 with only 3 points. FDXB still converges there.

Example row: case109272

Path	FE	RK2	BE
{0, 0.05, 1}	fail	fail	5 / 3
{0, 0.05, 0.1, 1}	fail	5 / 4	4 / 3
{0, 0.05, 0.15, 1}	5 / 4	4 / 3	4 / 3
{0, 0.05, 0.1, 0.2, 1}	4 / 4	4 / 3	4 / 3
{0, 0.1, 1} K = 2e-3	fail	fail	5 / 3

Numbers: NR iterations / FDXB iterations.

Effect of the time step on the 109,272-bus grid

exp08_fig5.py — BE homotopy, then NR, bus 6

- Five constant time steps: $dt = 1.0, 0.5, 0.25, 0.125$ and 0.1 .
- First step is always backward Euler at $t_1 = 0.005$, $K = 1e-4$.
- Each run runs the homotopy, then hands the last point to NR.
- Track voltage magnitude and voltage angle at bus 6 through every iteration.
- NR converges to $V = 1.0045$ pu and angle = 21.1 degrees for every dt .
- Smaller dt costs more LU factorizations without a better final answer.
- So the time step is a cost knob, not an accuracy knob for the final root.

Final answer for every dt

1.0045 pu

voltage magnitude at bus 6

21.1°

voltage angle at bus 6

Matches the paper for every time step tested.



Why change the fixed step size

improvements/path.py — follow_path_richardson

- The paper picks one dt_0 for the whole path. This forces a hard compromise.
- A small dt is safe but wastes LU factorizations.
- A large dt saves LUs but can blow up on the hardest grids.
- The best dt depends on the grid and even on the point along the path.
- Idea: estimate the local error from a Richardson step and let it steer the step size.
- Extrapolate the two half steps to gain one order for the kept point.
- Stop early if the path costs more than $\text{max_lu} = 150$ LU factorizations.

The Richardson trick

one full step

```
x_full = step(x, t, dt)
```

two half steps

```
half = step(x, t, dt/2)
```

```
both = step(half, t+dt/2, dt/2)
```

error and update

```
err = ||both - full|| / (2^p - 1)
```

```
x_new = both + (both - full) / (2^p - 1)
```

```
dt_new = dt * (tol / err)^(1 / (p + 1))
```

The step control loop, step by step

Every attempt costs three underlying integrator calls

- 1 Start at $t = 0$.**
 Take the paper's first step to $t_1 = dt_0$ with backward Euler.
- 2 Attempt one full step of size dt .**
 And two half steps of size $dt / 2$. Count three LUs.
- 3 Estimate the error.**
 $err = ||x_{both} - x_{full}||$ divided by $2^p - 1$, where p is the method order.
- 4 Accept if $err \leq tol$.**
 Store the Richardson extrapolation $x_{both} + (x_{both} - x_{full}) / (2^p - 1)$.
- 5 Rescale for the next step.**
 $dt_{new} = dt \cdot \text{clamp}(0.9 \cdot (tol/err)^{1/(p+1)}, 0.1, 4)$.
- 6 Stop early.**
Dynamic homotopy power flow
 If the total LU count exceeds $max_lu = 150$, mark the run as failed.

Order p for each integrator

Method	Order p
Forward Euler	1
Backward Euler	1
RK2	2
RK4	4

Fallback

If any of the three trial steps is non-finite, halve dt by four and try again.

What Richardson control achieves

170 runs = 34 grids × 5 settings of dt0 and K, correct root only

151 / 170

runs solved with Richardson only

156 / 170

with Hasbi's optimal multiplier

127 / 170

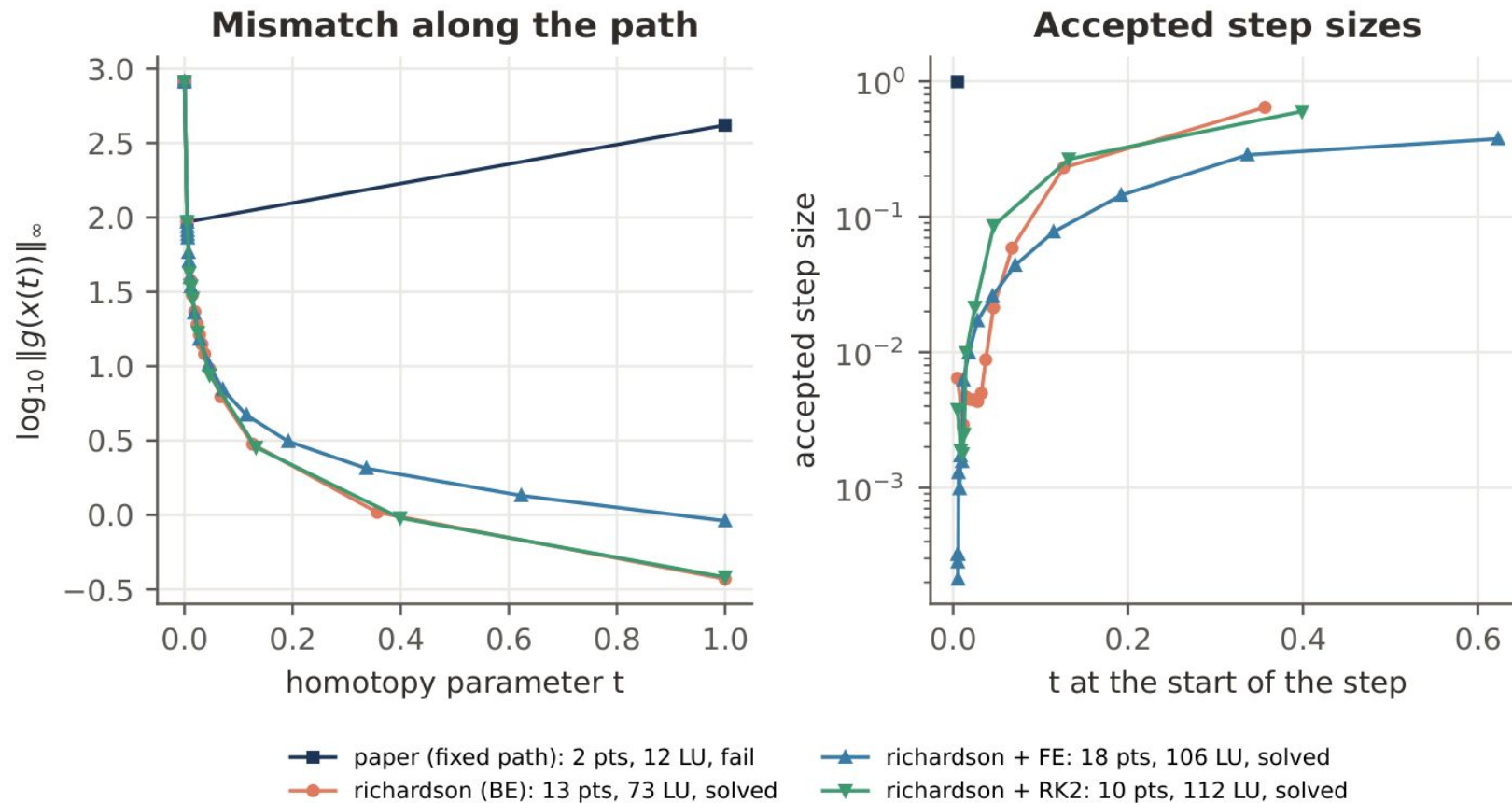
paper's method (baseline)

Findings unique to Richardson control

- Only change that solves case6024 at the paper's own setting ($dt0 = 0.005$, $K = 1e-4$).
- Solves case7092, case10595 and case12110 with a weak K where the paper's method diverges.
- Under error control, backward Euler stays the cheapest integrator: 10.6 LUs per case, against FE 37, RK2 48, RK4 87.

Richardson paths on case6024

richardson_path.pdf — mismatch and accepted step size for BE, FE, RK2



What the figure shows

- Left column: paper's fixed step.
- Right column: Richardson steps.
- Top row: mismatch $\|g\|$ along t .
- Bottom row: accepted dt vs t .
- BE runs are always safe.
- FE and RK2 need small dt near $t = 0$.

A second homotopy that did not help

improvements/homotopies.py — class ScaledHomotopy

- The paper's easy function shifts every equation by the same K.
- Idea from ref. 29: shift each equation by its own Jacobian diagonal.
- Define $g_0(x) = K D (x - x_0)$, where $D = |\text{diag } J(x_0)|$ scaled to mean 1.
- So a strong row gets a strong shift and a weak row gets a weak shift.
- Result: 129 of 170 runs solved. Almost the same as the paper's 127.
- Kept in the code as a documented negative result.

```
class ScaledHomotopy(FixedPointHomotopy):
    def __init__(self, problem, x0, K, counter=None):
        super().__init__(problem, x0, K, counter)
        d = np.abs(problem.jacobian(x0).diagonal())
        d[d == 0] = d[d > 0].min()
        self.scale = d / d.mean()

    def G(self, x, t):
        return (t * self.problem.g(x)
                + (1 - t) * self.K * self.scale * (x -
self.x0))

    def Gt(self, x):
        return self.problem.g(x) - self.K*self.scale*(x -
self.x0)
```

Gauss elimination and LU with partial pivoting

improvements/gauss.py — dense, $O(n^3)$, inner loops vectorised

- `gauss_solve` does partial pivoting on each column.
- The multiplier `m` eliminates the sub-diagonal in one row update.
- `back_substitution` walks up the upper triangle.
- `lu_factor` stores `L` (unit diagonal) and `U` in one packed array (Doolittle).
- `forward_substitution` then `back_substitution` give `lu_solve`.
- `factorize` wraps this with the same signature as `dynhomotopy.linalg.factorize`.
- So `NR` and `FDXB` can drop it in with no other change (Hasbi's `scratch.py`).
- Both algorithms agree with NumPy to about $1e-13$.

```
def lu_factor(a):
    lu = np.array(a, dtype=float)
    n = lu.shape[0]
    perm = np.arange(n)
    for k in range(n - 1):
        p = k + np.argmax(np.abs(lu[k:, k]))
        if lu[p, k] == 0: raise SingularError(...)
        if p != k:
            lu[[k, p]] = lu[[p, k]]
            perm[[k, p]] = perm[[p, k]]
        lu[k+1:, k] /= lu[k, k]
        lu[k+1:, k+1:] -= np.outer(lu[k+1:, k],
                                   lu[k, k+1:])
    return lu, perm
```

Why the paper needs a sparse LU

exp15_scratch_lu.py — same iterations, very different time

0.04 s

SciPy sparse LU (SuperLU)

424 s

our dense LU from scratch

14×

faster than repeating Gauss for 20 rhs

Same numerical answer

- Solvers give the same NR and FDXB iteration counts on 69- to 2000-bus grids.
- Largest difference in the returned voltages: about $5e-12$.
- So the from-scratch code is numerically correct.

Why FDXB factorizes only once

- One LU factor. Twenty right-hand sides. $14 \times$ faster than repeating Gauss elimination.
- The 424 s figure shows dense $O(n^3)$ also ignores sparsity.
- This confirms the design of the project's LU counter as the cost metric.

One command to run any switch on any grid

improvements/__main__.py — python -m improvements ...

```
$ python -m improvements case18482 --corrector
case18482: dt0 = 0.005, K = 0.0001, refiner = NR
```

method	outcome	path steps	NR its	LUs
paper	solved	2	3	5
corrector	solved	2	3	6

```
$ python -m improvements case6024 --multiplier --richardson
$ python -m improvements case36964 --dt0 0.1 --K 2e-3 \
    --corrector --multiplier

$ python -m improvements spectrum      case18482
$ python -m improvements feasibility  case6024
$ python -m improvements tune         case6024
$ python -m improvements scratch      case_ACTIVSg500limit \
    --refiner FDXB
```

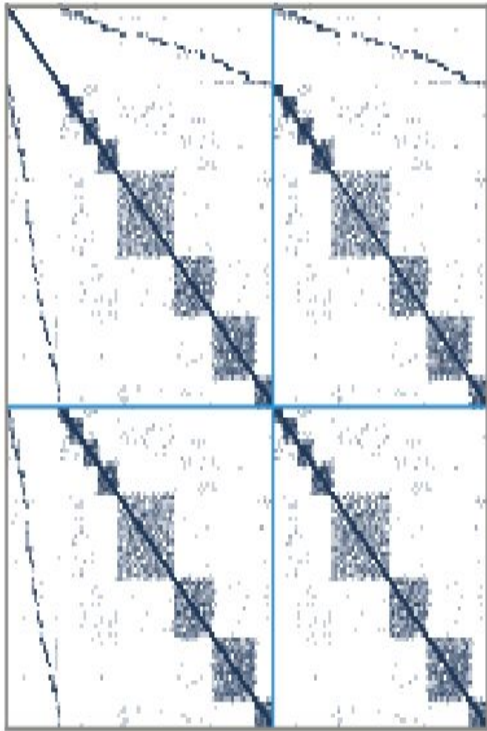
Switches available

- --corrector: Newton corrector (Apurbo)
- --multiplier: optimal step (Hasbi)
- --adaptive: halve when mismatch grows
- --richardson: error-controlled dt
- --step: FE, RK2, RK4, BE, BE-chord
- --homotopy: fpv, scaled, newton
- --refiner: NR or FDXB
- --dt0, --K: paper's parameters

Sparsity of the Jacobian at three grid sizes

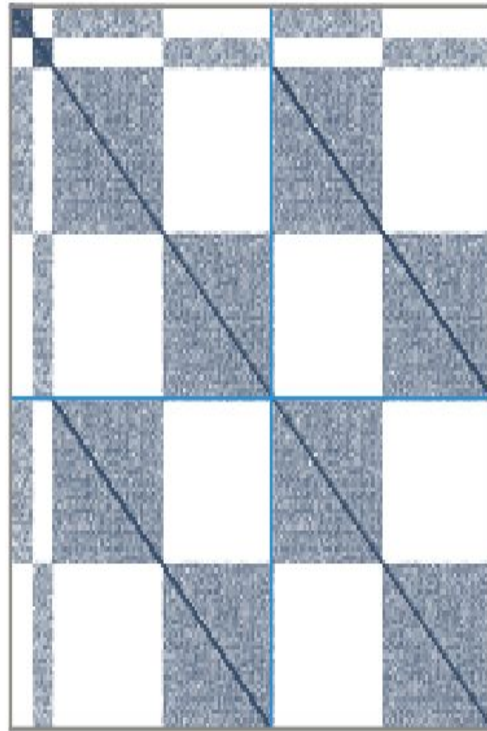
report/figures/spy.pdf — why sparse LU is not optional

case2000limit



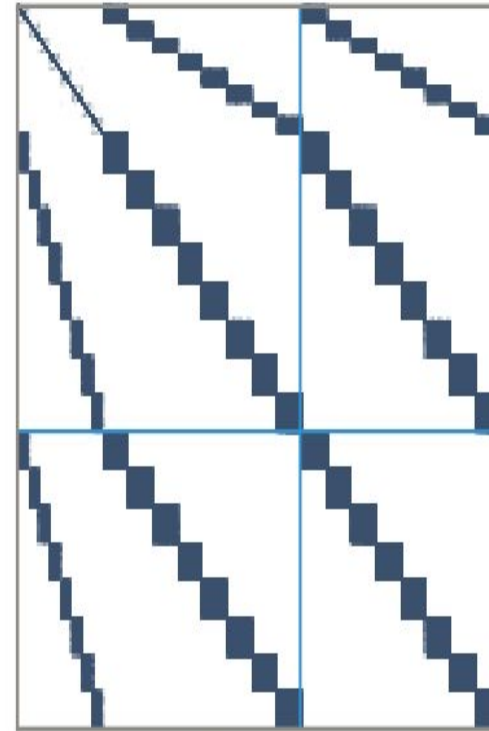
$n = 3,607$ $\text{nnz} = 26,345$

case18482



$n = 34,073$ $\text{nnz} = 258,947$

case109272



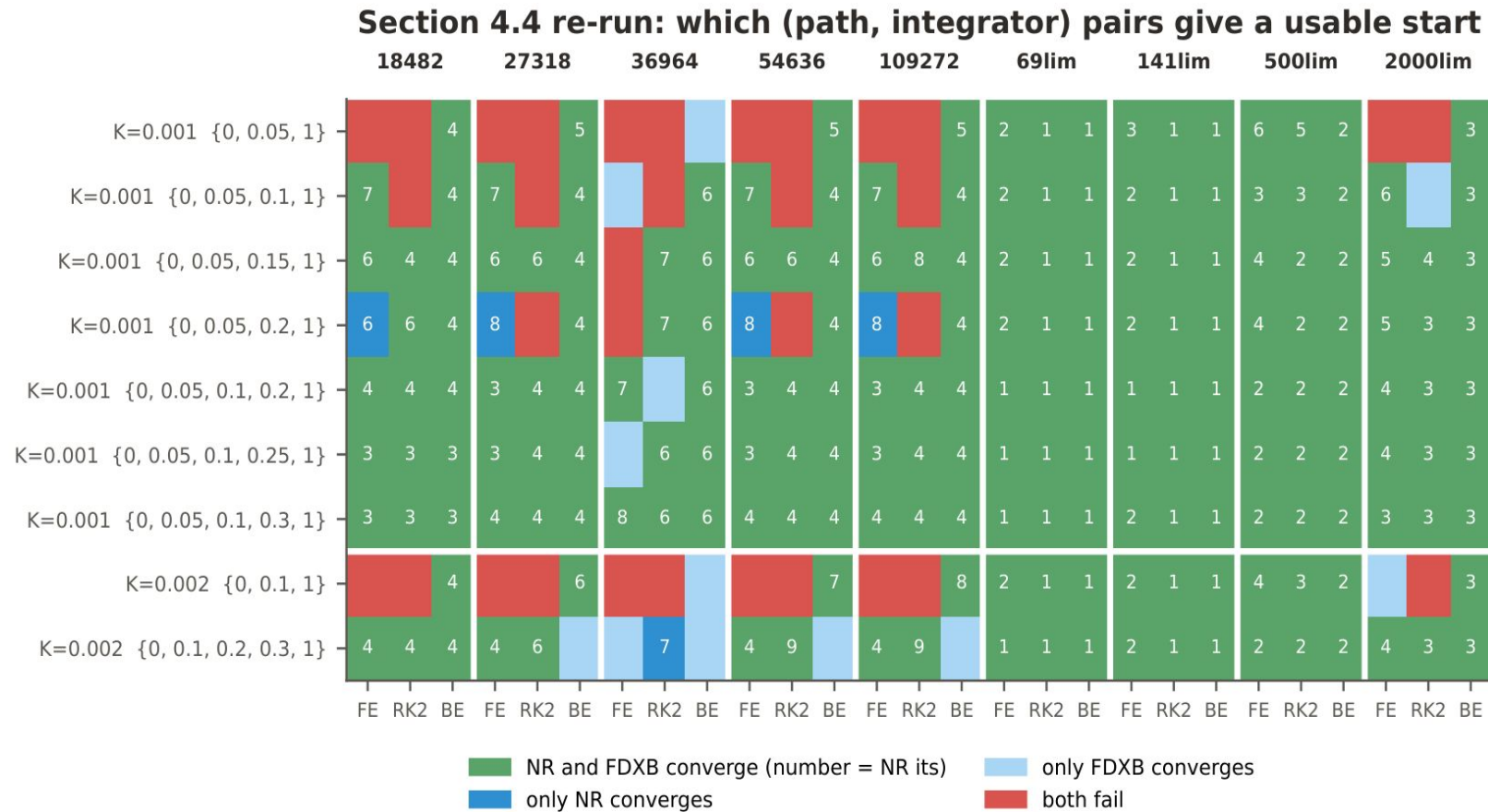
$n = 185,807$ $\text{nnz} = 1,397,667$

Reading the panels

- Left: 2000-bus grid.
- Middle: 18,482-bus grid.
- Right: 109,272-bus grid.
- Each dot: one non-zero of J .
- Density stays near 0.01 %.
- Every column has few entries.
- This is why splu is fast.

Section 4.4 rerun as one grid

report/figures/sec44_grid.pdf — paths × integrators, coloured by convergence



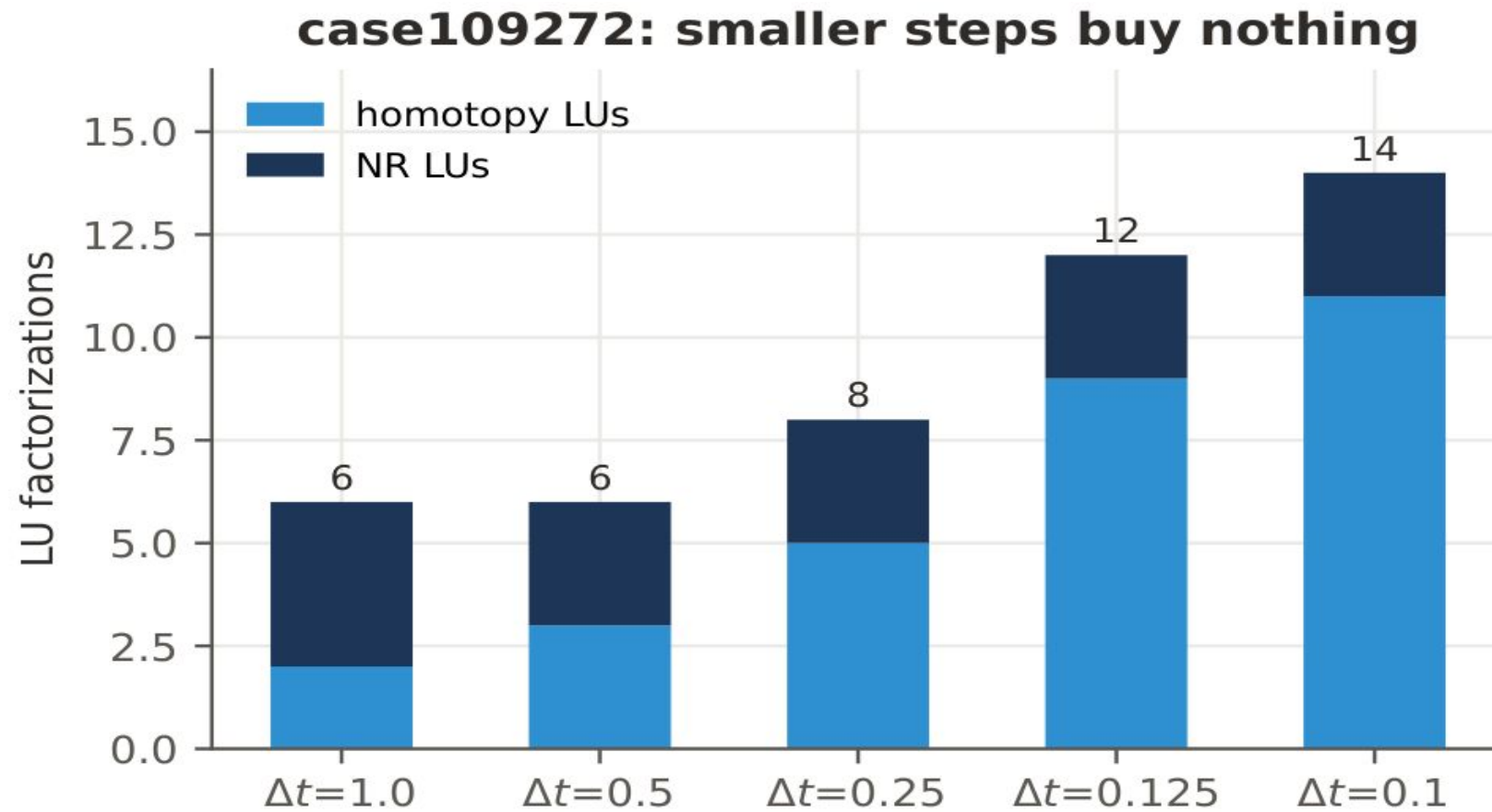
Colour legend

- NR and FDXB converge
- only NR converges
- only FDXB converges
- neither converges

Numbers inside each cell: NR iterations. BE is the only integrator that fills the grid.

Cost of a small time step on case109272

report/figures/fig5_tradeoff.pdf — homotopy vs NR LU factorizations



What the bars show

- Bar height: total LUs.
- Dark: homotopy LUs.
- Light: NR refinement LUs.
- Smaller dt: taller dark bar.
- NR bar hardly changes.
- Same final V and angle.
- So small dt costs more for the same result.

Tests I wrote

24 tests in the project. My share is below.

File	Test
tests/test_improvements.py	scaled homotopy: matches fpv when the diagonal is uniform
tests/test_improvements.py	Gauss elimination against numpy.linalg.solve
tests/test_improvements.py	LU factorization: L unit triangular, U upper triangular
tests/test_improvements.py	partial pivoting: swaps recorded in perm
tests/test_improvements.py	Richardson control: solves the 2x2 example
tests/test_cli.py	command-line tool: parses arguments and runs each study

All tests pass. Run with: `pytest -q`

Takeaways

Where my kernels moved the project forward

- **Honest cost.** Every LU is counted through one wrapper. So the paper's cost claim can be checked.
- **Richardson control.** Only change that solves case6024, case7092, case10595 and case12110.
- **From-scratch LU.** 424 s vs 0.04 s proves the sparse LU is essential at 2000 buses and above.
- **Command-line tool.** One line reproduces any experiment. Every switch is exposed.
- **Report figures.** Sparsity, Richardson paths, Section 4.4 grid and Figure 5 tradeoff.

Thank you